**FICHE 9 — AVANCÉ**

# Piloter Claude Code sur Riemann_Lab : contexte, vérification, plan, sessions

**Objectif.** Savoir conduire une session Claude Code longue (calcul, correctif C, run distribué) sans perte de qualité, et savoir prouver que le travail est juste. Suite des fiches 1 à 8 (PDF du 11/10/2026).

**Source officielle unique :** « Best practices for Claude Code », code.claude.com/docs/en/best-practices (lue le 11/11/2026), traduite et résumée. Tout ce qui n'en vient pas est marqué *[application Zeta]* ou *[déduit]*. Les commandes et options évoluent vite : vérifiez votre version (`claude --version`, `/help`).

## Réponse directe

Selon Anthropic, presque toutes les bonnes pratiques découlent d'**une seule contrainte** : la fenêtre de contexte se remplit vite, et les performances du modèle baissent à mesure qu'elle se remplit (consignes oubliées, erreurs plus fréquentes). Le contexte est donc **la ressource la plus importante à gérer**. La deuxième idée maîtresse : **donner à Claude un contrôle qu'il peut exécuter lui-même** (test, code retour, comparaison à une référence). Sans ce contrôle, « ça a l'air fini » est le seul signal, et c'est vous qui devenez la boucle de vérification.

## Les six principes officiels, appliqués au projet

| # | Principe (doc officielle, traduit) | Application Zeta [application Zeta] | Preuve dans Guide_Run_Zeta |
|---|---|---|---|
| 1 | Donner un moyen de vérifier : tests, build, comparaison à un fichier de référence. Exiger des **preuves** (sortie de commande, code retour) plutôt qu'une affirmation de succès. | Vos contrôles existent déjà : 59 tests de référence (16+9+34), Turing–Backlund COMPLET, 20/20 LMFDB, SHA-256 PC1/PC2, script de comparaison (annexe E). | Étiquettes « rapporté » / « vérifié » : un résultat de Claude Code sans sortie de commande reste « rapporté ». |
| 2 | Explorer, puis planifier, puis coder (mode plan : `Shift+Tab` jusqu'à « plan mode on », ou `claude --permission-mode plan`). Sauter le plan si le diff tient en une phrase. | Plan obligatoire pour D27 (`trap` dans `zeta_run.sh`), D29 (pivot pondéré), tout ce qui touche `illinois_arb.c`. Pas de plan pour une coquille. | Fiche 2.1 bis : hypothèse de Claude Code (« plancher dû à Z_rs ») contredite par ses propres mesures : explorer avant de corriger. |
| 3 | Contexte précis : nommer le fichier, le symptôme, l'endroit probable et ce que « réparé » veut dire. Référencer avec `@fichier`. | Ex. : « @scripts/zeta_run.sh : après Ctrl+C, services et turbo non restaurés (fiche 1.10). Ajoute un trap EXIT/INT/TERM. Réparé = 38 services et TURBO_STATUS=INACTIF après un Ctrl+C sur T=20000. » | Fiche 1.10 (b) : tableau des pannes = critère de réussite tout fait. |
| 4 | CLAUDE.md court : pour chaque ligne, « l'enlever ferait-il faire une erreur à Claude ? ». Sinon, la couper. Un CLAUDE.md trop long fait ignorer les vraies consignes. | Voir l'audit page suivante : `optimisation/CLAUDE.md` (MAJ 03/06/2026) contient des informations périmées. | E45 : tolérance 1e-12 absolue, inatteignable au-delà de t ≈ 16 384. |
| 5 | Gérer la session : `/clear` entre tâches sans lien ; après **deux** corrections ratées sur le même point, `/clear` et meilleur prompt ; `/compact <consignes>` ; `/rename` pour nommer les sessions. | Une session = un chantier nommé : `/rename v17-distrib-T1e6`, `/rename trap-zeta-run`. | E47 : deux sessions parallèles, même label F1 sur PC2, résultats écrasés. |
| 6 | Sous-agents pour enquêter (contexte séparé, résumé renvoyé) et pour une **relecture adversariale** du diff dans un contexte neuf. | « Utilise un sous-agent pour relire le diff de illinois_arb.c contre la fiche 2.1 bis ; ne signale que ce qui touche la justesse. » | E49 : NaN, chronomètre à virgule, sed cassé : erreurs trouvées après coup. |

## 1. Le contexte : pourquoi il faut le gérer

Le contexte contient **tout** : vos messages, chaque fichier lu, chaque sortie de commande. Une session de débogage ou d'exploration peut consommer des dizaines de milliers de tokens (doc officielle). *[application Zeta]* Un journal de run T=200000 relu en entier, ou un CSV de zéros affiché par erreur, remplit le contexte pour rien : demandez plutôt `grep -nE "Worker|RESULTATS|RESCAN|PROFIL"` (bonne pratique « Après » n°6 du guide) ou un résumé par un sous-agent.

| Outil (Claude Code) | Effet (doc officielle) | Quand l'utiliser dans Zeta |
|---|---|---|
| `/clear` | remet le contexte à zéro | entre « run distribué » et « rapport anglais » ; après 2 corrections ratées |
| `/compact Focus on …` | résume en gardant ce que vous indiquez | `/compact garde les fichiers modifiés, les commandes de test et les SHA` |
| `Esc` / `Esc Esc` ou `/rewind` | arrête / revient à un point de contrôle (conversation, code ou les deux) | dès que Claude part dans la mauvaise direction ; **ne remplace pas git** : les changements faits par Bash ne sont pas capturés |
| `/btw` | question à côté, la réponse n'entre pas dans l'historique | « /btw que veut dire ulp ? » pendant une longue session |
| `/context` | montre ce qui est chargé (CLAUDE.md compris) | vérifier que le bon CLAUDE.md est lu dans `c_modules/` |
| `claude --continue` / `--resume` | reprendre une session nommée | reprendre après une coupure (batterie du 03/11) |

## 2. Audit d'un CLAUDE.md réel du projet (exercice résolu)

Fichier : `src/calculs/optimisation/CLAUDE.md`, 124 lignes, MAJ 2026-06-03. Grille officielle « À inclure / À exclure » appliquée ligne par ligne. *[application Zeta, à confirmer par vous]*

| Contenu actuel | Verdict (grille officielle) | Pourquoi / correction proposée |
|---|---|---|
| « Précision cible C : PREC = 170 bits », exemple ctypes sur `illinois_mpfr.so` | À corriger : information **périmée** | Production = V17 avec `illinois_arb.so` (lsof, fiche 1.8 : `illinois_mpfr.so` non chargé). Remplacer par 1 ligne : module réellement utilisé. |
| « Répéter jusqu'à \|b−a\| < 1e-12 » ; tableau LMFDB « < 1e-12 » | À corriger : **faux** pour t > 16 384 | Fiche 2.1 bis : tolérance relative (≤ ~1 ulp). Une consigne fausse dans CLAUDE.md est relue à **chaque** session. |
| Règles de code C (mpfr_clear, MPFR_RNDN, pas de printf) | À garder | Règles propres au projet que Claude ne devinerait pas. |
| Workflow Git (branche Riemann_Lab_C) | À garder | « Repository etiquette » : explicitement dans la colonne À inclure. |
| « Pourquoi la Phase C : 80–90 % du temps » | À exclure ou dater | Chiffre qui change (profil V16 : détection 36 %, arb_C 23 %). Le mettre dans Handoff.md. |
| Prérequis apt, interface ctypes détaillée | À exclure | Claude les retrouve dans le Makefile et le code. |
| « Règle de double mise à jour — OBLIGATOIRE » | À transformer | Ce qui doit arriver **à chaque fois** relève d'un **hook** (déterministe), pas d'une consigne (indicative). |

**Règle officielle à retenir :** si Claude continue à faire ce qu'une règle interdit, le fichier est probablement trop long et la règle s'y perd. Mettre « IMPORTANT » sur **une** ligne seulement ; si tout est souligné, rien ne ressort. Ce qui ne sert que parfois va dans un **skill** (chargé à la demande), pas dans CLAUDE.md.

## 3. Consigne, hook ou skill ? Choisir le bon mécanisme

| Mécanisme | Nature (doc officielle) | Exemple Zeta [application Zeta] |
|---|---|---|
| CLAUDE.md | contexte persistant, **indicatif**, chargé à chaque session | « Toujours lancer les tests avec `-B -I` depuis ~/projet_zeta » |
| Hook | script exécuté à un moment précis, **garanti**, sans exception | existant : turbo au SessionStart. À envisager : bloquer `git add -A` (PreToolUse) ; Stop hook qui relance les 59 tests |
| Skill | connaissance ou procédure chargée **à la demande** | existant : riemann-lab, zeta-diagnostic. Candidat : « contrôles avant run distribué » (tableau 2.2 bis g) |
| Sous-agent | contexte séparé, outils limités, renvoie un résumé | relecteur « justesse numérique » en lecture seule (Read, Grep, Bash) |
| Permissions / sandbox | `/permissions` pré-approuve des commandes sûres | autoriser `git status`, `pytest` ; garder la confirmation pour `git commit` (votre règle STOP) |

## 4. Vérifier : quatre niveaux, du plus simple au plus solide

La doc officielle classe la vérification selon la force avec laquelle elle « bloque » la fin du travail :

| Niveau | Mécanisme officiel | Version Zeta |
|---|---|---|
| 1. Dans le prompt | demander d'exécuter le contrôle et d'itérer | « … puis lance tests_v17 et montre la sortie » |
| 2. Sur la session | `/goal` : condition réévaluée après chaque tour | « goal : Turing COMPLET et 0 écart > 2 ulp contre le CSV de référence » (à tester dans votre version) |
| 3. Barrière déterministe | Stop hook : bloque la fin du tour tant que le script échoue | script qui exécute les 3 fichiers de tests de référence |
| 4. Second avis | sous-agent vérificateur dans un contexte neuf | relecture du diff contre PLAN.md ; ne signaler que ce qui touche la justesse |

**Mise en garde officielle :** un relecteur à qui l'on demande des défauts en trouvera presque toujours, même si le travail est sain. Lui demander de ne signaler que ce qui touche la **justesse** ou les exigences, sinon on sur-construit (couches de défense inutiles, tests de cas impossibles).

## 5. Les cinq pièges officiels et leur trace dans le projet

| Piège (doc officielle) | Correctif officiel | Trace dans le projet |
|---|---|---|
| Session « fourre-tout » | `/clear` entre tâches sans lien | sessions mêlant run, rapport et monitoring [déduit des fiches 1.x] |
| Corriger encore et encore | après 2 échecs : `/clear` + meilleur prompt | E49 : corrections successives dans la même session |
| CLAUDE.md sur-spécifié | élaguer ; convertir en hook | audit section 2 |
| Écart confiance / vérification | toujours fournir une vérification ; sinon ne pas livrer | D28 : « 1e-12 » affiché, 1,7e-6 réel sur paires proches |
| Exploration infinie | borner l'enquête ou la confier à un sous-agent | lecture de 73 .md ou de journaux entiers |

## 6. Sessions parallèles sans collision

Doc officielle : plusieurs sessions en parallèle via **worktrees** (copies git isolées), application de bureau, ou sessions dans le cloud ; motif **Rédacteur / Relecteur** : une session écrit, une autre relit dans un contexte neuf. *[application Zeta]* Le risque chez vous n'est pas seulement le code : ce sont les **machines**. Deux sessions qui lancent des runs sur PC2 se partagent CPU, dossiers et labels (E47). Règle proposée : **une session par chantier, une seule session autorisée à lancer des calculs**, labels uniques (date + chantier).

## 7. Gabarit de prompt Claude Code (à copier)

```
Contexte  : Riemann_Lab, production V17, PC1 (riemann@zeta-lab).
Lis d'abord : @scripts/zeta_run.sh et Guide_Run_Zeta fiche 1.10.
Mode      : plan d'abord ; n'écris rien avant mon accord.
Tâche     : ajouter un trap EXIT/INT/TERM qui restaure services et turbo.
Hors champ: ne touche pas au calcul (compute_zeros_v17.py).
Réussi si : après Ctrl+C sur T=20000 -> 38 services (liste des noms
            identique), TURBO_STATUS=INACTIF, pas de runs/.actif.
Preuves   : montre chaque commande lancée et sa sortie.
Relecture : un sous-agent relit le diff ; justesse seulement.
STOP      : aucun git add / commit sans mon accord. Jamais git add -A.
```

Chaque ligne correspond à un principe : contexte précis (3), explorer/planifier (2), critère de réussite (1), preuves (1), relecture (6), règle STOP (permissions, section 3).

## 8. Exercice (30 min)

• Sur PC1, dans `~/projet_zeta/src/calculs/optimisation`, ouvrir Claude Code en mode plan (`claude --permission-mode plan`).

• Demander : « Audite ce CLAUDE.md avec la grille Include/Exclude des best practices. Pour chaque ligne : garder, corriger, déplacer (Handoff.md, skill, hook). Ne modifie rien. »

• Comparer sa réponse au tableau de la section 2. Noter les écarts.

• Lancer `/context` : quels CLAUDE.md sont chargés (racine, optimisation, c_modules) ?

• **STOP** : décider vous-même des lignes à changer, puis seulement après, faire appliquer et commiter.

## 9. Auto-évaluation

| Question | Réponse |
|---|---|
| Quelle est la ressource n°1 à gérer selon Anthropic ? | La fenêtre de contexte. |
| Que faire après deux corrections ratées sur le même point ? | `/clear` et un prompt meilleur qui intègre ce qu'on a appris. |
| Une action doit avoir lieu à chaque fois sans exception : CLAUDE.md ou hook ? | Hook (déterministe) ; CLAUDE.md est indicatif. |
| Quand sauter le mode plan ? | Quand le diff se décrit en une phrase. |
| `/rewind` annule-t-il un `rm` lancé par Bash ? | Non : seules les modifications faites par les outils d'édition sont capturées. |
| Pourquoi relire avec un sous-agent plutôt que dans la même session ? | Contexte neuf : il n'est pas biaisé par le raisonnement qui a produit le code. |

## Sources

Anthropic, « Best practices for Claude Code », https://code.claude.com/docs/en/best-practices (consulté le 11/11/2026). Exemples projet : Guide_Run_Zeta.md v0.19 (07/11/2026) ; src/calculs/optimisation/CLAUDE.md (03/06/2026). Fiches 1 à 8 : PDF « Fiches_Claude_Zeta_FR » (11/10/2026).